Experimental Report on the School of Computer and Artificial Intelligence at Zhengzhou University

Grade：2024 Major：SOFTWARE ENGINEERING Class：GROUP - 01

Name：MD TASNIMUL HASAN Student Number：202480090104

Instructor：ALEX Score： 2026 year 11 month 01 date

Experimental Topic：Intent Usage, Multi-Activity Programs, Event Handling.

1. Experimental Purpose

- Master the creation and use of explicit Intents to navigate between multiple activities within an app.
- Master the use of implicit Intents to invoke external apps (e.g., browser, dialer, camera).
- Understand data passing between activities using putExtra() and getIntent().
- Understand activity lifecycle and the result-returning mechanism via startActivityForResult() / ActivityResultLauncher.
- Master event handling in Android using both the anonymous inner class approach and the android:onClick XML attribute approach.

2. Experimental Content

1.Multi-Activity Program

- Create a main activity (MainActivity) and at least two additional activities (SecondActivity, ResultActivity).
- Navigate from MainActivity to SecondActivity using an explicit Intent.
- Pass data (e.g., a user's name and age) from MainActivity to SecondActivity via putExtra().
- Receive a result back from SecondActivity using registerForActivityResult() and display it in MainActivity

2.Intent Usage

- Explicit Intent: Navigate between the app's own activities.
- Implicit Intent: Launch the system browser, dialer, or share sheet from a button click.
- Optionally, use an Intent to open a web page (ACTION_VIEW) and to make a phone call (ACTION_DIAL).

3.Event Handling

- Implement **button click events** using anonymous inner classes (setOnClickListener).
- Implement **long-click events** using setOnLongClickListener and show a Toast.
- Implement **XML-based event handling** using android:onClick="methodName" on a button.
- Handle **EditText text-change events** using a TextWatcher to update a counter in real time.

3. Experimental Environment

Item	Specification
OS	Windows 10 / 11 (or macOS)
IDE	Android Studio (Giraffe / Hedgehog or later)
Language	Java
SDK	Android SDK API 24 – 34 (minSdk 24, targetSdk 34)
Emulator/Device	Android Emulator (Pixel, API 30+) or physical device
Build System	Gradle

4. Procedure

Step 1: Create the Project

1. Open Android Studio and create a **New Project** using the **Empty Views Activity** template.
2. Set the project name to IntentEventDemo, choose **Java** as the language, and set the minimum SDK to **API 24**.
3. Wait for the Gradle sync to complete.

Step 2: Design the Main Layout (activity_main.xml)

1. Add a vertical LinearLayout as the root container.

2. Place the following views:

- An EditText (id: etName) with hint "Enter your name".
- An EditText (id: etAge) with hint "Enter your age" and inputType="number".
- A TextView (id: tvCharCount) showing "Characters: 0" — used for the TextWatcher demo.
- A Button (id: btnGoSecond) labeled "Go to Second Activity (Explicit Intent)".
- A Button (id: btnOpenBrowser) labeled "Open Browser (Implicit Intent)".
- A Button (id: btnDial) labeled "Dial a Number (Implicit Intent)".
- A Button (id: btnXmlClick) labeled "XML onClick Demo" with android:onClick="onXmlButtonClick".
- A Button (id: btnLongPress) labeled "Long-Press Me".
- A TextView (id: tvResult) to display the result returned from SecondActivity.

Step 3: Create the Second Activity Layout (activity_second.xml)

1. Create a new layout with a vertical LinearLayout.
2. Place:

- A TextView (id: tvWelcome) to display the name and age passed from MainActivity.
- An EditText (id: etFeedback) to let the user type a feedback message.
- A Button (id: btnSendBack) labeled "Send Result Back".

Step 4: Create the Second Activity Java Class

1. Create a new Java class SecondActivity extending AppCompatActivity.
2. In onCreate(), call setContentView(R.layout.activity_second).
3. Retrieve the passed data using getIntent().getStringExtra("name") and getIntExtra("age", 0), then display them in tvWelcome.
4. On btnSendBack click:

- Read the feedback text from etFeedback.
- Create an Intent and put the feedback as an extra (putExtra("feedback", text)).
- Call setResult(RESULT_OK, intent) and finish() to return to MainActivity.

Step 5: Register the Second Activity in the Manifest

1. Open AndroidManifest.xml.
2. Add <activity android:name=".SecondActivity" /> inside the <application> tag.
3. Optionally, add android:exported="false" for security.

Step 6: Implement Intent and Event Handling Logic in MainActivity

1. **TextWatcher demo**: Attach a TextWatcher to etName; in afterTextChanged(), update tvCharCount to show the current character count.
2. **Explicit Intent + result**: In onCreate(), register an ActivityResultLauncher<Intent> using registerForActivityResult(new ActivityResultContracts.StartActivityForResult(), ...). In the callback, if resultCode == RESULT_OK, read the "feedback" extra and display it in tvResult.
3. **Explicit Intent button**: On btnGoSecond click:

- Validate that name and age are non-empty.
- Create an Intent(MainActivity.this, SecondActivity.class).
- Put the name and age as extras using putExtra().
- Launch it with launcher.launch(intent).

4.**Implicit Intent — Browser**: On btnOpenBrowser click, create new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.android.com")) and call startActivity(). Wrap in a try-catch to handle no browser installed.

5.Implicit Intent — Dialer: On btnDial click, create new Intent(Intent.ACTION_DIAL, Uri.parse("tel:12345")) and call startActivity().

6.XML onClick handler: Define a public method onXmlButtonClick(View view) in MainActivity and show a Toast. This is wired to the button via android:onClick in the XML — this demonstrates the XML-based event handling approach.

Long-click event: Attach setOnLongClickListener to btnLongPress and show a Toast "Long press detected!". Return true to consume the event.

Step 7: Run and Test the Application

1. Click **Run** to install the app on the emulator or device.
2. Verify the following:

- Typing in the name field updates the character counter live.

- Clicking **Go to Second Activity** navigates to SecondActivity, which shows the passed name and age.
- Typing feedback in SecondActivity and clicking **Send Result Back** returns to MainActivity and displays the feedback in tvResult.
- Clicking **Open Browser** opens the system browser at android.com.
- Clicking **Dial a Number** opens the dialer app with the number pre-filled.
- Clicking **XML onClick Demo** shows a Toast (verifies XML-based event handling).
- Long-pressing the long-press button shows a Toast.

3.Check logcat for any errors; verify that pressing the **Back** button returns to MainActivity correctly.

5. Results and Analysis

Results

- The explicit Intent successfully launched SecondActivity and passed the name and age as extras.
- getStringExtra and getIntExtra correctly retrieved the data and displayed it in SecondActivity.
- The ActivityResultLauncher correctly received the feedback from SecondActivity and displayed it in MainActivity after setResult().
- Both implicit Intents (browser and dialer) launched the appropriate external apps.
- TextWatcher updated the character count in real time.
- The android:onClick XML-based handler, the anonymous setOnClickListener, and setOnLongClickListener all worked as expected.

Analysis

- Explicit vs Implicit Intent: Explicit Intents name the exact target component (used for internal navigation), while implicit Intents declare an action and let the system resolve a suitable app (used for browser, dialer, sharing). Implicit Intents can fail if no app handles the action — hence try-catch or resolveActivity() checks are good practice.
- startActivityForResult vs ActivityResultLauncher: startActivityForResult() is deprecated in favor of the modern registerForActivityResult() API, which is lifecycle-aware and less error-

prone.

- Event handling approaches: Three approaches were demonstrated — (a) anonymous inner class (setOnClickListener), (b) XML android:onClick, and (c) long-click listener. The XML approach keeps code cleaner for simple actions but has weaker compile-time safety. The listener approach is more flexible and testable.
- TextWatcher: This is the standard way to observe EditText changes; the callback fires after each keystroke, enabling live UI updates.
- Manifest requirement: SecondActivity must be declared in AndroidManifest.xml; otherwise, the explicit Intent throws ActivityNotFoundException at runtime.

Lifecycle note: When navigating to SecondActivity, MainActivity enters onPause() / onStop(); upon return, it resumes. The ActivityResultLauncher callback fires only when the result is delivered, respecting lifecycle order.

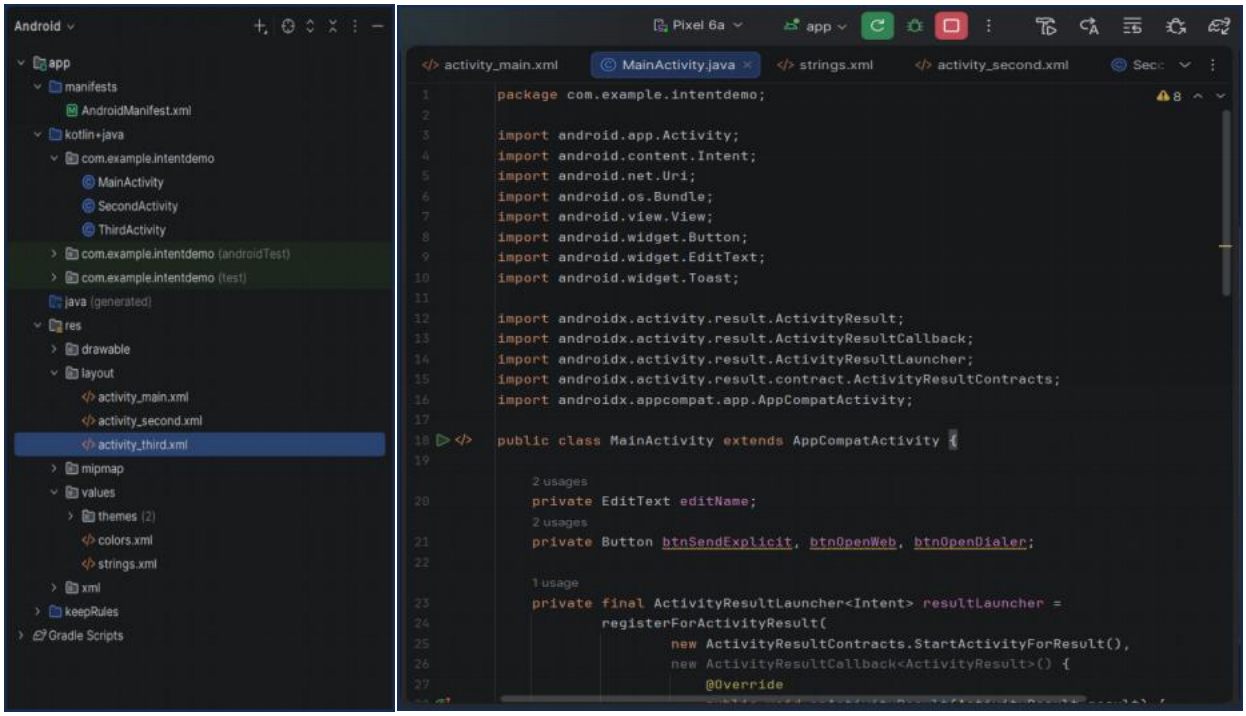


Figure 1: Project Structure and MainActivity.java Code

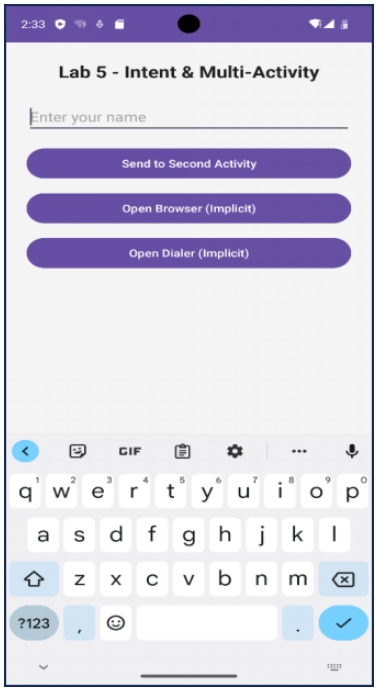


Figure 2: Main Screen

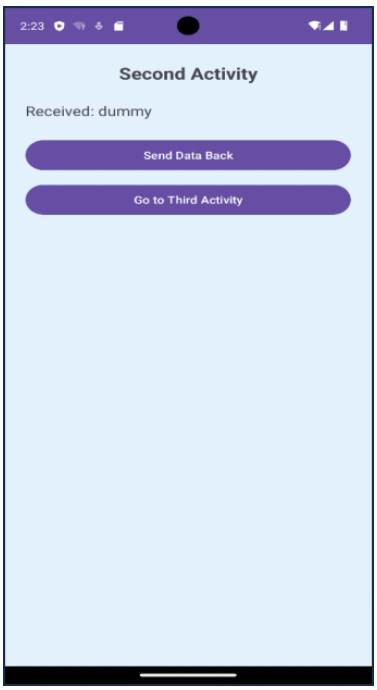


Figure 3: Second Activity

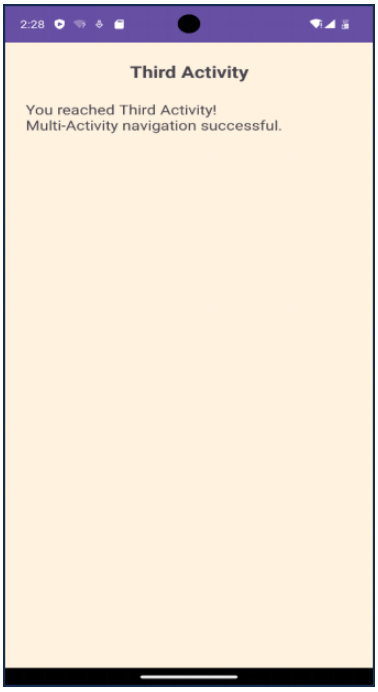


Figure 4: Third Activity

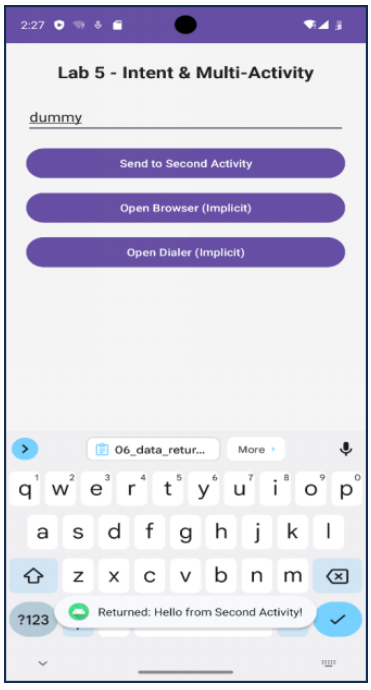


Figure 5: Data Returned to Main

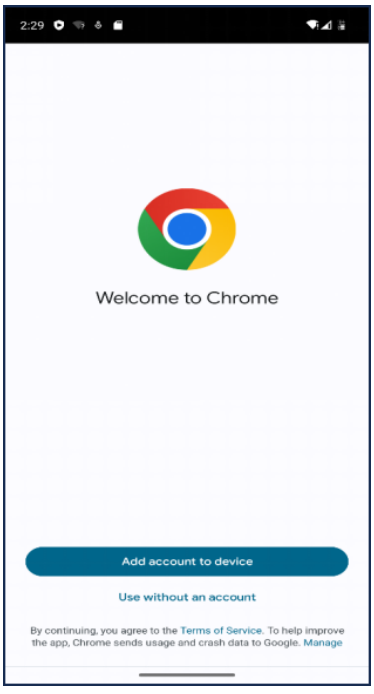


Figure 6: Browser opened



Figure 7: Dialer opened

6. Summary

In this experiment, I practiced three core Android concepts:

1. Intents — Used explicit Intents for internal activity navigation and implicit Intents for invoking the browser and dialer. Learned to pass data with putExtra() and retrieve it with getStringExtra() / getIntExtra().
2. Multi-Activity Programs — Created MainActivity and SecondActivity, registered them in the manifest, passed data forward, and returned a result using setResult() and ActivityResultLauncher. Understood the activity lifecycle implications of navigation.
3. Event Handling — Implemented click, long-click, and text-change events using anonymous inner classes, XML android:onClick, and TextWatcher. Compared the three approaches and their trade-offs.

Key takeaways: Intents are the backbone of Android communication — both within an app and across apps. The modern result-returning API (registerForActivityResult) is safer than the deprecated startActivityForResult. Multiple event-handling approaches exist, and choosing the right one depends on simplicity vs. flexibility requirements. The experiment strengthened my ability to build multi-screen apps with data flow and interactive event handling.